Foundations of Computing I
Summary

Simon Schuhmacher

UZH FS2026

Contents

1 Propositional Logic **1**
Basic Logical Equivalences . . . 1
Commutative Law . . . 1
Associative Law . . . 1
Distributive Law . . . 1
Identity Law . . . 1
Negation Law . . . 1
Double Negation Law . . . 1
Idempotent Law . . . 1
Universal Bound Law . . . 1
De Morgan's Law . . . 1
Absorption Law . . . 1
Negations of **t** and **c** . . . 1
XOR . . . 1
Implication (If-Then) . . . 1
Equivalence (Biconditional If and Only if) . . . 1

2 Digital Logic Circuits **1**
Disjunctive Normal Form (DNF) . . . 1
Conjunctive Normal Form (CNF) . . . 1
NOR Gate . . . 2
NOT Using NOR . . . 2
OR Using NOR . . . 2
AND Using NOR . . . 2
NAND Gate . . . 2
NOT Using NAND . . . 2
OR Using NAND . . . 2
AND Using NAND . . . 2

3 Induction and Recursion **2**
Proof by Mathematical Induction . . . 2
Proof by Strong Mathematical Induction . . . 2
Method of the Loop Invariant to Prove Algorithm Correctness . . . 2

4 Functions and Convolution **3**
Laws of Exponents . . . 3
Laws of Logarithms . . . 3
1D Convolution . . . 3
Convolution Table Approach . . . 3
Properties of Convolution . . . 3
Why We Flip the Function in the Definition of Convolution . . . 4
Dirac Function . . . 4
2D Convolution . . . 4
Box Filters . . . 4
Example Low-Pass Filter . . . 4
Example High-Pass Filter . . . 4
Example Horizontal (x) Axis Filter . . . 4
Example Vertical (y) Axis Filter . . . 4
Boundary Issues . . . 4

Filters of Even Side Length 5

5 Relations and Cryptography **5**
Reflexivity, Symmetry, and Transitivity 5
Transitive Closure 5
The Relation Induced by a Partition 5
Equivalence Classes of an Equivalence Relation 5
Congruence Modulo n 5
Properties of Congruence Modulo n 6
Finding an Inverse Modulo n 6
Bézout's Theorem 6
Euclidean Algorithm 6
RSA Cryptography 6
Bézout's Identity 6

1 Propositional Logic

Basic Logical Equivalences

Commutative Law

- $p \wedge q \equiv q \wedge p$
- $p \vee q \equiv q \vee p$

Associative Law

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
- $(p \vee q) \vee r \equiv p \vee (q \vee r)$

Distributive Law

- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Identity Law

- $p \wedge \mathbf{t} \equiv p$
- $p \vee \mathbf{c} \equiv p$

Negation Law

- $p \vee {\sim}p \equiv \mathbf{t}$
- $p \wedge {\sim}p \equiv \mathbf{c}$

Double Negation Law

- ${\sim}({\sim}p) \equiv p$

Idempotent Law

- $p \wedge p \equiv p$
- $p \vee p \equiv p$

Universal Bound Law

- $p \vee \mathbf{t} \equiv \mathbf{t}$
- $p \wedge \mathbf{c} \equiv \mathbf{c}$

De Morgan's Law

- ${\sim}(p \wedge q) \equiv {\sim}p \vee {\sim}q$
- ${\sim}(p \vee q) \equiv {\sim} \wedge {\sim}q$

Absorption Law

- $p \vee (p \wedge q) \equiv p$
- $p \wedge (p \vee q) \equiv p$

Negations of t and c

- ${\sim}\mathbf{t} \equiv \mathbf{c}$
- ${\sim}\mathbf{c} \equiv \mathbf{t}$

XOR

- $p \oplus q \equiv (p \vee q) \wedge {\sim}(p \wedge q)$
- $p \oplus q \equiv (a \wedge {\sim}b) \vee ({\sim}a \wedge b)$

Implication (If-Then)

- $p \to q \equiv {\sim}p \vee q$

Equivalence (Biconditional If and Only if)

- $p \leftrightarrow q \equiv (p \to q) \wedge (q \to p)$

2 Digital Logic Circuits

Disjunctive Normal Form (DNF)

"OR of ANDs"

From truth table to DNF

1. Identify the rows whose output is 1
2. AND together all variables in the same row (negate the variable when 0)
3. OR together the 1-rows

Conjunctive Normal Form (CNF)

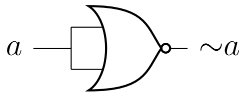
"AND of ORs"

From truth table to CNF

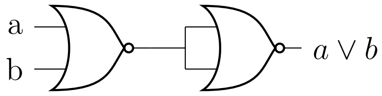
1. Identify the rows whose output is 0
2. OR together all variables in the same row (negate the variable when 1)
3. AND together the 0-rows

NOR Gate

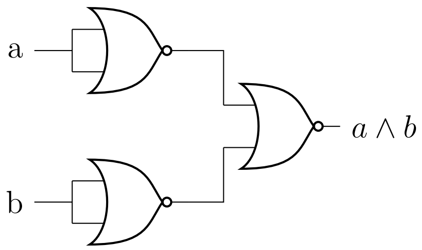
NOT Using NOR



OR Using NOR

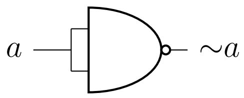


AND Using NOR

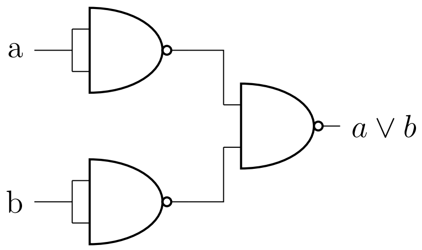


NAND Gate

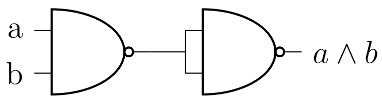
NOT Using NAND



OR Using NAND



AND Using NAND



3 Induction and Recursion

Proof by Mathematical Induction

To prove that a property $P(n)$ is **true** for every integer $n \geq b$:

1. **Base Case**: Show that $P(b)$ is true
2. **Inductive Hypothesis**: Assume $P(k)$ true for any $k \geq b$
3. **Inductive Step**: **Show** that $P(k) \to P(k+1)$ **is true**
4. **Conclusion**: If $P(b)$ is true and $P(k) \to P(k+1)$ is true, conclude that $P(n)$ is true for all $n \geq b$

Proof by Strong Mathematical Induction

- Prove recursive problems with several base cases (e.g., recurrence relations with order larger than 1)
- Makes a stronger hypothesis than the principle of Mathematical Induction:
  - The **base step** may contain proofs for **several initial values**, rather than just the base case
  - In the inductive step, the trust of $P(k)$ is assumed not just for an arbitrary $k \geq b$ but for **all values through** k

Let b, b be two integers with $a \leq b$. To prove that a property $P(n)$ is **true** for every integer $n \geq a$:

1. **Base Case**: Show that $P(n)$ is true for all base cases from a to b, that is: $P(a) \wedge P(a+1) \wedge \cdots \wedge P(b)$
2. **Inductive Hypothesis**: For any $kgeqb$, assume $P(i)$ **is true for** $i = a, \ldots, b, \ldots, k$
3. **Show** that $P(k+1)$ **is true**
4. **Conclusion**: If 1 and 2 are verified, conclude that $P(n)$ is true for all $n \geq a$

Method of the Loop Invariant to Prove Algorithm Correctness

For every $n \geq 0$, let $P(n)$ be the loop invariant for a loop. **If the following four properties are true, then the loop is correct** with respect to its pre- and post-conditions:

1. **Basic property**: the loop invariant is true before the loop starts, i.e., $P(0)$ is true before the loop starts. ($P(0)$ is $P(n)$ when $n = 0$). → "The loop invariant is true before the loop starts (i.e., base case is true)"
2. **Inductive property**: for any integer $k \geq 0$, if **both the guard G and the loop invariant** $P(k)$ are true before an iteration of the loop, then $P(k+1)$ is true after iteration. → "It remains true after every iteration (i.e., $P(k) \to P(k+1)$)"
3. **Eventual Falsity of the Guard**: after a finite number of iterations of the loop, the guard G becomes false. → "The loop eventually stops (i.e., that the loop condition eventually turns false)"
4. **Correctness of the post-condition**: if N is the number of iterations after which G is false and $P(n)$ is true, then the post-condition is satisfied. → "At the point when the loop ends, you need to verify that the **loop invariant** is true"

4 Functions and Convolution

Laws of Exponents

If b and c are any positive real numbers and u and v are any real numbers, the following laws of exponents hold true:

- $b^u b^v = b^{u+v}$
- $(b^u)^v = b^{uv}$
- $\frac{b^u}{b^v} = b^{u-v}$
- $(bc)^u = b^u c^u$

Laws of Logarithms

For any positive real numbers b, c and x with $b \neq 1$ and $c \neq 1$:

- $\log_b(xy) = \log_b x + \log_b y$
- $\log_b(\frac{x}{y}) = \log_b x - \log_b y$
- $\log_b(x^a) = a \log_b x$
- $\log_c x = \frac{\log_b x}{\log_b c}$

1D Convolution

$f(n)$, $g(n)$, $y(n)$: $\mathbb{Z} \to \mathbb{R} \quad \forall n \in \mathbb{Z}$

1D convolution between f and g, denoted with $y(n) = (f * g)(n) = \sum_{k=-\infty}^{+\infty} f(k)g(n-k)$

- f: input function
- g: kernel or filter

To convolve two functions f and g: $f * g$

1. Flip g horizontally ($-k$)
2. Slide it over f by adjusting n and compute the sum of the products, starting at the first overlap
3. Continue until the last overlap
4. Calculate $y(n)$ for using all n-values used to slide g over f

Note: Since the operation is commutative, sometimes it might be more convenient to compute $g * f$

Convolution Table Approach

1. Create vector representations of f and g
2. Identify *min index* and *max index* → min and max non-zero values on the x-axis for f and g
3. Create a table with f as columns from *min index* to *max index* and g as rows from *min index* to *max index*
4. Multiply the values
5. Create $y(n)$ by summing the diagonals
6. $n = 0$ at the diagonal where *row index* + *column index* $= 0$ (and equivalent for other n-values)

Properties of Convolution

- **Commutative Law**: $f * g = g * f$
- **Associative Law**: $(f * g) * h = f * (g * h)$

- **Associative Law with Scalar Multiplication**: $(\lambda f) * g = f * (\lambda g) = \lambda(f * g)$
- **Distributive Law**: $f * (g + h) = (f * g) + (f * h)$
- **Convolution with a Dirac**: $f * \delta = f$

Why We Flip the Function in the Definition of Convolution

$(f * g)(n) = (g * f)(n) \Rightarrow \sum_{k=-\infty}^{+\infty} f(k)g(n-k) = \sum_{k=-\infty}^{+\infty} g(k)f(n-k)$

- Without flipping, convolution would not be commutative

Dirac Function

Also called *impulse function.* The result of the convolution of a function $f(n)$ with a Dirac is the function itself: $(f * \delta)(n) = f(n)$

$$\delta(n) = \begin{cases} 1 & n = 0 \\ 0 & \text{elsewhere} \end{cases}$$

or

$$\delta(n-i) = \begin{cases} 1 & n = i \\ 0 & \text{elsewhere} \end{cases}$$

Note that $f(n) * \delta(n-i)$ shifts $f(n)$ by i to the right.

2D Convolution

Let F, G, Y be **functions** defined on the infinite set $\mathbb{Z}^2$: $F(i,j)$, $G(i,j)$, $Y(i,j)$: $\mathbb{Z}^2 \to \mathbb{R} \quad \forall (i,j) \in \mathbb{Z}^2$

We define the 2D convolution between f and g: $Y(i,j) = (F * G)(i,j) = \sum_{u=-\infty}^{+\infty} \sum_{v=-\infty}^{+\infty} F(u,v)G(i-u, j-v)$

- f: input image
- g: kernel or filter

Box Filters

1. Flip the filter in both dimensions (= 180 deg turn)
2. Slide the filter over the image, starting in the top right
3. Compute the sum of products (replace each pixel with a weighted sum of its neighbors)
4. Continue towards the right until the end of the row
5. Move to the next row, starting from the left again

Note: If we don't want to change the intensities, the sum of all elements in the filter should be 1.

Example Low-Pass Filter

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Example High-Pass Filter

0	1	0
1	-4	1
0	1	0

Example Horizontal (x) Axis Filter

-1	0	1
-1	0	1
-1	0	1

Example Vertical (y) Axis Filter

-1	-1	-1
0	0	0
1	1	1

Boundary Issues

- **Zero-padding** of the input image boundaries to achieve the **same size** of the input and output image.

- **No zero-padding** of the input image results in a **smaller size** of the output image.

Filters of Even Side Length

- If our filter has even side length, we need to determine the center out of 4 options. By convention, we chose the point at the bottom right (option 4)
- If we add zero-padding, this needs to be adapted according to where the center is chosen

	1	2	
	3	4	

5 Relations and Cryptography

Reflexivity, Symmetry, and Transitivity

Let R be a relation on a set A

1. R is **reflexive** if, and only if, for all $x \in A, x\ R\ x$
2. R is **symmetric** if, and only if, for all $x, y \in A$, ***if*** $x\ R\ y$ then $y\ R\ x$
3. R is **transitive** if, and only if, for all $x, y, z \in A$, ***if*** $x\ R\ y$ and $y\ R\ z$ then $x\ R\ z$

A relation that satisfies all these three properties is called an **Equivalence Relation**.

Transitive Closure

To make a relation transitive that does not contain certain **ordered pairs**, we need to add some ordered pairs. The relation R^t obtained by adding the **minimum number of ordered pairs** to ensure transitivity is called the **transitive closure** of the relation.

The Relation Induced by a Partition

- A **partition** of a set A is a collection of mutually disjoint subsets A_i whose union is A
- Given a partition, the relation **induced** by the partition of A is a relation where all the elements within each subset A_i of the partition are related to one another and themselves
- A relation induced by a partition is always **reflexive**, **symmetric**, and **transitive**, therefore it is always an **equivalence relation**

Equivalence Classes of an Equivalence Relation

- Given an equivalence relation on a certain set A and any particular element a in A, the subset $[a]$ of all elements related to a under R is called the **equivalence class** of a:
$[a] = \{x \in A \mid x\ R\ a\}$
- **Distinct equivalence classes** of an *equivalence relation* form a **partition** of A
- Any element of an equivalent class is called **Class Representative**
- E.g., for $\{0, 3, 4\}, \{1\}, \{2\}$, the **distinct equivalence classes** of R are:
$[0], [1], [2]$ (or alternatively $[3], [1], [2]$ or $[4], [1], [2]$)
    - $[0] = [3] = [4]$ because they represent the same equivalence class

Congruence Modulo n

Let a, b, and n be any integers and suppose $n > 1$. The following statements are all equivalent

1. $n \mid (a - b)$
2. $a \equiv b \bmod n$
3. $a = b + kn$ for some integer k

4. a and b have the same (nonnegative) remainder when divided by n

5. $a \bmod n = b \bmod n$

Properties of Congruence Modulo n

1. $(a + b) \bmod n =$ $[(a \bmod n) + (b \bmod n)] \bmod n$

2. $(a - b) \bmod n =$ $[(a \bmod n) - (b \bmod n)] \bmod n$

3. $(a \cdot b) \bmod n =$ $[(a \bmod n) \cdot (b \bmod n)] \bmod n$

4. $(a^b) \bmod n = (a \bmod n)^b \bmod n$

5. $-a \bmod b = b - (a \bmod b)$

If you first perform addition, subtraction, multiplication of integers and then **reduce the result modulo n**, you obtain the same result as when the operation is performed on the reduced version of the integers.

Finding an Inverse Modulo n

The modular inverse of an integer a is an integer x such that:

- $a \cdot x \equiv 1 \pmod{n}$
- That means: $(a \cdot x) \bmod n = 1$
- *Example:* Compute the inverse of 3 modulo 7: We want to determine x such that $3x \equiv 1 \pmod{7}$. An inverse is $5 \equiv 1 \pmod{7}$. Other possibilities are e.g., 12, -2.
- The **inverse** of an integer a modulo n exists if and only if a and n are **relatively prime**.
- Two integers a and b are **relatively prime** if and only if $\gcd(a, b) = 1$

Bézout's Theorem

If a and b are positive integers, then there exist integers s and t such that $\gcd(a, b) = \boldsymbol{s}a + \boldsymbol{t}b$

Euclidean Algorithm

Let A and B be two integers with $A > B \geq 0$, Euclid's algorithm is defined by the following recursive function:

$$\gcd(A, B) = \begin{cases} A & \text{if } B = 0 \\ \gcd(B,\ A \bmod B) & \text{if } B > 0 \end{cases}$$

$A \bmod B$ denotes the **remainder** of the division of A by B.

RSA Cryptography

- The plaintext M is converted into a ciphertext C: $C = M^e \bmod S$
- S and e are the **public keys**, i.e., anyone can use them to encrypt their messages
- The plaintext M for a ciphertext C is then recovered: $M = C^d \bmod S$
- Where d is the **private key**, i.e., it is secret, and only the recipient knows it
- For RSA to work $M = (M^e)^d \bmod S$ must hold for all positive integers $M < S$
- This holds if:
  - $S = p \cdot q$ where p and q are **prime numbers**
  - e and the product $(p-1)(q-1)$ are **relatively prime**, i.e., $\gcd(e, (p-1)(q-1)) = 1$
  - d is a **positive inverse** of $e \bmod (p-1)(q-1)$, i.e., $ed \equiv 1 \pmod{(p-1)(q-1)}$
- RSA works because it is easy to generate two large prime numbers p and q, but it is hard to factor their product S into pq

Bézout's Identity

For any non-zero integers a and b with $\gcd(a, b) = d \quad \exists x, y$ (Bézout coefficients) such that $ax + by = d$